

Übungsblatt 1

Punkte:

--

 / 20

Fortgeschrittene Algorithmen (Wintersemester 2025/26)

Bearbeitet von: Matthias Janßen, Raphael Thelen, Tillmann

Faust

October 22, 2025

Aufgabe 1

```
MaxProben(int[] [] A, int m, int n):
    //Merkn aus welcher Richtung der Rover kommt
    char previous[m][n]
    //Matrix durchlaufen
    for i = 1 to m do
        for j = 1 to n do
            //Startfeld braucht keine Aktion
            if(i == 1 & j == 1) then skip
            //Am linken Rand kann der vorherige Punkt nur oben liegen
            else if(i == 1) then
                //Die Summe des vorherigen und aktuellen Knoten wird gespeichert,
                //so wird der gesamte Wert des Weges gezählt
                A[i][j] = A[i][j] + A[i][j-1]
                previous[i][j] = '0'
            //Am oberen Rand kann der vorherige Punkt nur links liegen
            else if(j == 1) then
                A[i][j] = A[i][j] + A[i-1][j]
                previous[i][j] = 'L'
            else
                //Je nachdem ob die Summe des Weges von "oben" oder "links" und
                //des aktuellen Knoten größer ist wird der Vorgänger
                // und Wert gespeichert
                max = A[i][j] + A[i-1][j]
                if A[i][j] + A[i][j-1] > max then
                    max = A[i][j] + A[i][j-1]
                    previous[i][j] = 'L'
                else
                    previous[i][j] = '0'
                A[i][j] = max
    //Pfad rekonstruieren indem von hinten nach vorne gegangen wird
    path = [] // Stack
```

```

while not (m == 1 & n == 1) do
  if previous[m][n] == 'L' then
    n = n - 1
    path.push('o') // 'o': Osten
  else
    m = m - 1
    path.push('s') // 's': Süden
return path, A

```

Der Algorithmus läuft in $\mathcal{O}(m \cdot n)$ Zeit, da jedes Feld der Matrix genau einmal besucht wird.

Aufgabe 2

a)



Figure 1: Gegenbeispiel (a)

Der Greedy Algorithmus aus Aufgabe a wählt die erste Aufgabe da sie die kürzeste ist. Dadurch kann nur eine Aufgabe gewählt werden, auch wenn es möglich wäre 2 Aufträge zu erledigen.

c)

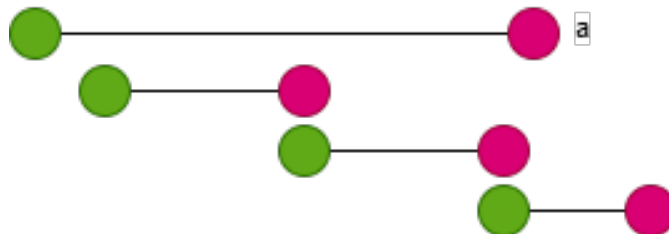


Figure 2: Gegenbeispiel (c)

Der Algorithmus wählt Auftrag a da dieser den frühesten Startzeitpunkt hat. Dadurch kann nur ein Auftrag ausgewählt werden, obwohl 3 möglich wären.

Aufgabe 3

a)

```
GreedyTimer(int[] t, int z)
    L = { }
    while z > 0
        for m = 0 to t.length do
            if t[m] > z
                L.add[t[m-1]]
                z = z - t[m-1]
                m = 0
            if m == t.length
                L.add[t[m]]
                z = z - t[m]
                m = 0
    return L
```

Der Algorithmus wählt immer die größte Taste aus die in die verbleibende Zeit passt. Da alle Tasten außer 1 den Teiler 5 teilen wird so immer der optimale Weg gefunden.

b)

Ein Pizzaofen mit den Tasten 1, 5 & 6. Für eine Eingabe von 10 Sekunden würde ein Greedy Algorithmus, der immer die größtmögliche Taste wählt, die Tastenkombination 6, 1, 1, 1, 1 auswählen anstatt 5, 5.